

Team Roles and Contributions Report (April 13 –April 26, 2026)

Summary

Over the past bi-weekly development cycle, the system has achieved a comprehensive leap from underlying infrastructure to its core application form. Below is an in-depth summary of the four core achievements of this phase:

1. Core Product Form Leap

During this period, the product completed its most crucial paradigm shift, fully upgrading from providing single, static relationship analysis reports to a two-way "Virtual Figure" interactive platform equipped with continuous memory and emotional coherence.

- **Immersive Multi-Turn Conversation Loop:** Completely abandoned the rigid Question-and-Answer (QA) model found in traditional LLM applications, pivoting to a continuous dialogue system based on streaming and dynamic context. The system can now accurately capture the flow of user intent in complex conversations and deliver highly anthropomorphic responses using a specific virtual persona.
- **Websocket Stable Long-Connection & Debounce Mechanism:** To resolve the user experience bottlenecks of traditional HTTP/SSE in high-frequency interactions, we fully introduced two-way Websocket communication, significantly enhancing the naturalness and immersion of the interaction.

2. System Orchestration & Workflow Deep Refactoring

To address the low execution efficiency and tight module coupling in the legacy linear workflow, this phase performed a "major surgery" level, large-scale refactoring of the core LangGraph pipeline.

- **Graph Decoupling and Single Responsibility:** Vertically dismantled the originally bloated, monolithic execution graph into two core graphs with clear responsibilities: `ContextGraph`, focusing on data aggregation and context preparation, and `AnalysisGraph`, focusing on logical reasoning and content generation.
- **Massive Latency Reduction via Multi-Branch Parallelization:** By introducing a Router distribution mechanism, we achieved the parallel execution of three core data pipelines within `ContextGraph`—basic profile extraction, vector database recall, and MBTI knowledge retrieval—which are then merged at an aggregation node. This directly broke through the previous performance bottleneck of up to 1 minute and 32 seconds, resulting a qualitative leap in system response speed.

3. RAG & Memory Engine Upgrade

As user data accumulates, the system demands extremely high retrieval precision and context purity. This cycle completed crucial engine upgrades in memory retrieval and state persistence.

- **Three-Dimensional Composite Scoring Engine Incorporating Time Decay:** Discarded the early, crude retrieval method that solely relied on "vector cosine distance." The brand-new recall engine integrates three major dimensions: **semantic similarity**, **business weight parameters**, and a **time decay factor based on a half-life formula**. This upgrade allows the system to act like a human brain, achieving highly timely and precise recall.
-

Leishiyu Chen

I. Weekly Summary

During this period, I led the evolution of the backend architecture towards a highly dynamic "Virtual Figure real-time interaction" scenario, shifting our focus from static analysis to fluid user engagement. I completed the field expansion of the core database and the development of associated APIs, and overcame the experience bottleneck of traditional HTTP Q&A in the virtual figure scenario by independently developing a Websocket communication mechanism based on timers and message batching. I also standardized and refactored the project directory structure to better support future multi-agent scalability.

II. Key Progress

- **Core Data Flow Expansion:** Added `words_to_user` and `words_from_user` fields to the Crush table to precisely capture both parties' speaking habits and tone, which significantly improves the LLM's ability to mirror the target persona accurately. Persisted `context_block` in the RelationChain table to provide ready-to-use context for virtual figure conversations, drastically reducing query overhead.
- **Websocket Interaction Gateway Construction:** Deprecated the traditional SSE push and established two-way Websocket communication. Designed a "15-second waiting period" debounce batching strategy: resetting the timer during user input intervals, and batching multiple scattered messages to the Agent for processing once the timer ends. This prevents the system from rudely interrupting the user mid-thought, while implementing a simulated random delay of 0.8s-2.2s for replies to mimic natural human typing cadence.
- **State Refresh API Development:** Developed the `/recalculateContextBlock` API. When the system records new Events, Chat Topics, or updates profiles, it can actively trigger a recalculation and refresh of the relationship context, ensuring the real-time accuracy of the virtual figure's memory and strictly preventing outdated context from causing conversational hallucinations.
- **Project Directory Architecture Refactoring:** Reorganized the project structure of the `agent/graph` directory based on Graph types, improving the code maintainability of large workflows.

III. Next Steps

- **Persistence Mechanism Refinement:** Establish a robust mechanism for saving and retaining raw context data in the database (e.g., user-uploaded chat screenshots, original narrative text) for future model fine-tuning and auditability.
 - **Network Stability Assurance:** Introduce a Heartbeat mechanism and automatic reconnection logic to the Websocket gateway to ensure long-connection stability under poor network conditions.
 - **Data Entry Expansion:** Explore and provide multi-dimensional ways to supplement relationship and profile context, and complete the development of corresponding APIs to enrich the core data lake.
-

Yihan Liu

I. Weekly Summary

During this period, my core task was to tackle "memory management" and "prompt engineering" refactoring in complex conversational scenarios. I deeply investigated and resolved the critical short-term memory pollution issue encountered when combining LangGraph with PostgresSaver, effectively preventing context overflow. I also completed the State planning for the `VirtualFigureGraph`, and introduced cutting-edge industry architectural concepts to prepare for the next phase of refactoring.

II. Key Progress

- **Short-term Memory (Checkpoint) Troubleshooting & Fixes:** Upon introducing PostgresSaver, I discovered that the graph's State carried a large number of useless intermediate products (like the ORM instance `Entities`), causing serialization failures and severe Token consumption. By refactoring the State structure, I strictly stripped intermediate states from `ContextGraphState` and `AnalysisGraphState`, establishing a streamlined snapshot strategy that "only saves core Requests, Contexts, and LLM Outputs," which drastically reduced API overhead costs.
- **Continuous Analysis Logic Decoupling:** Based on the repaired short-term memory, I implemented the `/continuousAnalysis` API. Without needing to re-invoke the heavily time-consuming `ContextGraph`, it can retain the full picture of the previous analysis round simply by loading the Checkpoint, achieving seamless context inheritance for follow-up user queries.
- **VirtualFigureGraph State & Memory Design:** Designed an independent workflow State for the virtual figure. Defined `Messages` using the append mode of `Annotated[List[BaseMessage], add_messages]`, and established the standard that "system prompts do not enter the memory bank and are built separately in real-time each round," ensuring dynamic context injection.
- **Multi-Agent & Advanced Prompt Research:** Deeply analyzed the open-source project DeerFlow, extracted the best engineering practices of Meta Prompt and Multi-Agent (Supervisor) architectures, and established a clear optimization direction for the subsequent mental model, specifically focusing on dynamic task routing.

III. Next Steps

- **Prompt System Refactoring:** Utilize Meta Prompt concepts to refactor the full Agent prompt system, and dynamically inject the current timestamp into the Virtual Figure's system prompt to enhance real-time awareness.
 - **Long-term Memory Hub Upgrade:** Advance the integration and debugging of the Volcengine Mem0 long-term memory framework to replace or supplement the existing vector retrieval memory.
 - **Multi-Agent Architecture Implementation:** Collaborate with the team to discuss and push the Virtual Figure architecture towards a robust multi-agent collaboration model.
-

Haojie Hu

I. Weekly Summary

During this period, I completely rewrote the initial version of the vector recall strategy to address critical issues of context irrelevance and memory hallucination during extended dialogues. Upgraded from a crude recall solely relying on "vector distance" to a highly sophisticated, composite recommendation-level recall engine. By seamlessly integrating "semantic distance, dynamic weight, and time decay," this new architecture ensures the LLM retrieves not just related information, but the most accurate, contextually appropriate, and timely long-term memory, thereby drastically improving the natural flow and logical consistency of the Virtual Figure's interactions.

II. Key Progress

- **Time Decay Engine for Memories:** Introduced a time decay algorithm conforming to the human forgetting curve. For entities with timestamps like Events and ChatTopics, I calculated the time difference and applied the formula `exp(-delta_days / HALF_LIFE_DAYS)`. This ensures the weight of old, potentially outdated memories smoothly decreases over time, accurately mirroring how recent relationship events naturally hold more conversational relevance than distant past occurrences.
- **Composite Scoring & Ranking Strategy:** Established a rigorous recall scoring formula: `score = (semantic_score * 0.8 + weight * 0.2) * time_decay`. By comprehensively considering text similarity alongside an entity's inherent importance weight and time decay, this calculation prevents generic semantic matches from overshadowing critical user facts, achieving a highly optimal and balanced sorting of candidate memories.
- **Source Range Normalization & Business Filtering:** Implemented refined business isolation at the vector recall layer. For different recall types (e.g., FROM_EVENT, FROM_KNOWLEDGE), I strictly validated the `relation_chain_id` and applied exclusive filtering conditions. This not only blocks mismatched noise data but also enforces strict data isolation, ensuring context never bleeds across different relationship chains.
- **Structured Memory Output:** Refactored the data assembly logic post-retrieval. The pipeline now supports the automatic extraction and merging of entities' `.toJson()` results, weights, and timestamps immediately after recall. This directly provides high-density, cleanly structured context for the downstream Agent, significantly reducing token consumption and minimizing the LLM's parsing burden.

III. Next Steps

- **Intelligent Recall Judgement:** Optimize system logic using lightweight routing strategies to enable the Agent to intelligently judge whether it truly needs to trigger long-term memory (DB or Mem0) recall based on the exact semantics of each conversation round, thereby drastically reducing unnecessary database retrieval overhead and system latency.
 - **Performance Tuning & Evaluation:** Conduct rigorous query latency stress testing and optimize `pgvector` index structures for high-concurrency scenarios. This will ensure the system maintains rapid response times even after the introduction of the new complex scoring and mathematical sorting operations.
-

Tian Bu

I. Weekly Summary

During this period, I completed a massive structural refactoring of the LangGraph workflows to resolve critical performance bottlenecks. I dismantled the originally bloated linear execution chain into parallel multi-branch flows, and successfully overcame the technical challenge of manually managing LLM Tool Calls within deterministic StateGraph nodes. This pivotal shift guarantees strict state control and achieves highly customized external tool invocation without sacrificing execution stability.

II. Key Progress

- **ContextGraph Multi-Branch Refactoring:** Deprecated the previously highly inefficient linear execution (which previously took a staggering 1m 32s). Introduced a dynamic Router mechanism to refactor the `ContextGraph`, achieving parallel execution of three core chains (basic context generation, vector DB recall, and MBTI knowledge & interaction signal extraction), and completed seamless data aggregation immediately before the `nodeOrganizeContext` node to drastically reduce overall generation latency.
- **AnalysisGraph Tool Call Manual Takeover:** Abandoned the potentially out-of-control automatic scheduling of standard ReAct Agents to eliminate unpredictable infinite loops. I encapsulated a custom tool interceptor `ToolAndItsArgsHandler` and parsing executor `handleIfToolCall` for precise, granular intervention.
- **Dynamic Parameter Hijacking & Injection:** Addressing the critical pain point that the LLM cannot stably output the required `relation_chain_id`, I intercepted the Tool Call request directly at the graph node. I forcefully injected the absolute correct ID from the backend system context into the `useKnowledge` tool parameter, ensuring the strict accuracy of external data extraction and preventing unauthorized data overlap.
- **Visualization & Performance Analysis Integration:** Completed the configuration of `langgraph.json` in the project root directory, and successfully introduced LangSmith (`langgraph dev`) for local Studio graph structure visual debugging and pipeline latency analysis.

III. Next Steps

- **Tool Encapsulation & Generalization:** Collaborate with Haojie to standardize and encapsulate more underlying capabilities like "vector database recall" as Tools, allowing the LLM to call them on demand when needed to conserve tokens.
- **Supervisor Orchestration R&D:** Drawing on newly learned architectural concepts, explore and refactor the existing full Agent workflows to support more complex, multi-layered decision routing.
- **Debugging Infrastructure Rollout:** Guide team members to fully integrate LangGraph Studio debugging infrastructure, significantly improving the workflow troubleshooting efficiency for the entire team.